📚 Algorithm Design & Analysis

Brief Overview

This note covers **Algorithm Design & Analysis** and was created from a 593-page PDF. The document dives into algorithmic problem solving, providing clear pseudocode conventions, rigorous correctness proofs, and a detailed analysis of algorithm efficiency.

Key Points

- Pseudocode conventions and flowchart history
- **Correctness proofs** for classic algorithms (e.g., Euclid's GCD)
- Time and space analysis, including amortized reasoning
- Fundamental data structures and sorting/searching techniques

---

📘 Fundamentals of Algorithmic Problem Solving

🖊️ Pseudocode Conventions

- **Assignment** is written with an arrow **←**.
- **Comments** start with double slashes **//**.
- **Indentation** indicates the scope of control structures such as for, if, and while.
- Variable declarations are omitted for brevity.

> *Pseudocode* is a high-level, language-independent description of an algorithm that is close enough to modern programming languages for easy comprehension.

📊 Flowchart Note

- Historically, **flowcharts** used geometric shapes to represent steps.
- They are now rarely used except for very simple algorithms.

💻 From Specification to Implementation

- An algorithm's description (natural language or pseudocode) must be **converted into a program** written in a specific programming language.
- This program is considered the **implementation** of the algorithm.

✅ Proving an Algorithm's Correctness

- Correctness means the algorithm produces the required result for **every legitimate input** within a finite amount of time.
- **Common technique: Mathematical induction** (leveraging the natural sequence of iterations).
- Example: **Euclid's algorithm** for GCD
  - Relies on the equality **$\gcd(m, n) = \gcd(n, m \bmod n)$**.
  - The second integer **decreases** each iteration.
  - Terminates when the second integer becomes **0**.
- **Tracing** with specific inputs is **insufficient** for a proof; a single counterexample **disproves** correctness.
- For **approximation algorithms**, correctness involves proving that the **error** does not exceed a predefined bound.

📈 Analyzing Algorithms

Desired Qualities

Quality	Description
Correctness	Produces the right output for all valid inputs.
Time efficiency	How fast the algorithm runs (measured by counting the basic operation).
Space efficiency	Extra memory required beyond the input.
Simplicity	Easier to understand, implement, and debug; often aesthetically pleasing.
Generality	- **Problem generality**: solves a broader class of problems. - **Input generality**: accepts a natural set of inputs for the problem.

- **Simplicity** can lead to fewer bugs and sometimes better performance, though not always.

- **Generality** trade-offs:
  - Solving a broader problem (e.g., computing **gcd**) can simplify solving a narrower one (checking relative primality).
  - Unnecessary generality may waste effort (e.g., sorting to find a median).

> *"A designer knows he has arrived at perfection not when there is no longer anything to add, but when there is no longer anything to take away."* – Antoine de Saint-Exupéry

Coding an Algorithm

- Transitioning from a correct specification to a working program may introduce **bugs**; careful implementation is essential.

📂 Important Problem Types

🔤 Sorting

> *Sorting* is the process of rearranging items of a list into **nondecreasing order** (requires a **total ordering** relation).

- **Keys**: the attribute used to order records (e.g., name, student number, GPA).
- **Why sort?**
  - Produces ordered outputs (rankings, directories).
  - Facilitates **searching** (binary search, dictionaries).
  - Serves as a preprocessing step for many algorithms (geometric, compression, greedy techniques).
- **Algorithmic landscape**:
  - Best comparison-based sorts achieve $\approx n \log_2 n$ comparisons.
  - No single sorting algorithm dominates all scenarios.
- **Key properties**

Property	Meaning
Stability	Preserves the relative order of equal keys.
In-place	Uses only $O(1)$ extra memory (aside from a few variables).

🔎 Searching

> *Searching* finds a **search key** in a set (or multiset).

- **Algorithms range** from simple **sequential search** to efficient **binary search** (requires sorted data) and advanced data-structure-based methods.
- **No universal best algorithm**: trade-offs involve speed vs. memory, and suitability for dynamic data (frequent insertions/deletions).
- **Dynamic considerations**: In environments where data changes often, choose structures that balance **search**, **insertion**, and **deletion** costs (e.g., balanced trees, hash tables).

📚 Fundamental Data Structures

📏 Arrays

> An **array** stores elements contiguously in memory and allows **constant-time** access via an index.

- Index ranges: typically **0 … n-1** (or **1 … n**).
- Used as building blocks for other structures (strings, matrices).

🧩 Linked Lists

Type	Description
Singly linked list	Nodes contain **data** and a **single pointer** to the next node.
Doubly linked list	Nodes contain pointers to **both successor and predecessor**.
Header node (optional)	Stores metadata (e.g., length) and pointers to first/last nodes.

- **Access time**: $O(k)$ to reach the k-th node (must traverse from the head).

- **Advantages**: no need for pre-allocation; efficient **insertions** and **deletions** by pointer adjustments.

📚 Linear Lists, Stacks, and Queues

Structure	Access Pattern	Typical Use
Stack	**LIFO** (push/pop at one end – *top*)	Recursive algorithm support, expression evaluation.
Queue	**FIFO** (enqueue at *rear*, dequeue at *front*)	Scheduling, breadth-first search, buffering.
Linear list	General sequence supporting search, insert, delete	Foundation for higher-level structures.

🏆 Priority Queues & Heaps

- **Priority queue** stores elements from a totally ordered universe, supporting **find-max**, **delete-max**, and **insert**.
- **Heap** implementation provides efficient priority-queue operations (typically $O(\log n)$).

📈 Graphs

> A **graph** $G = \langle V, E \rangle$ consists of a finite set of **vertices** V and a set of **edges** E .

- **Undirected edge**: unordered pair (u, v); vertices are **adjacent**.
- **Directed edge (arc)**: ordered pair (u, v); u is the **tail**, v the **head**.
- **Incidence**: an edge is **incident** to its endpoint vertices.
- Graphs may be labeled with letters, numbers, or strings for clarity.

📊 The Analysis Framework (Section 2.1)

Measuring Input Size

- Choose a **natural size metric** (e.g., number of elements n, matrix order).

Units for Running Time

- Count executions of the **basic operation** (the operation that dominates the cost).

Orders of Growth

- Focus on **asymptotic behavior** as input size → ∞ (e.g., $O(n)$, $\Theta(n \log n)$).

Case Analyses

Case	Description
Worst-case	Maximum cost over all inputs of size n .
Best-case	Minimum cost over all inputs of size n .
Average-case	Expected cost assuming a probability distribution on inputs.
Amortized	Average cost per operation over a sequence of operations on the same data structure.

- **Example**: Sequential search average comparisons
  - Successful search ($p = 1$): $(n + 1)/2$.
  - Unsuccessful search ($p = 0$): n .
- **Average-case analysis** often requires classifying inputs and assuming a probability model; it is more intricate than worst-case analysis.
- **Amortized efficiency** can reveal that occasional expensive operations are offset by many cheap ones (e.g., union-find with path compression).

Recap of Framework

- **Time efficiency** → count of basic operation executions.
- **Space efficiency** → count of extra memory units used.
- Distinguish **worst**, **average**, and **best** cases when performance varies with input.
- Primary interest: **order of growth** as input size grows without bound.

🧦 Probabilistic Analysis of the Sock Problem

> *Problem*: Ten socks consist of five matching pairs. Draw socks at random without replacement.

- **Best-case scenario** – the first two socks form a matching pair.
  - Probability: after any first sock, exactly one of the remaining nine socks matches it.
  - $P_{\text{best}} = \frac{1}{9}$.
- **Worst-case scenario** – the first five draws are all of different colours, so no pair appears until the sixth draw.
  - Probability that the first five socks are all different: $P_{\text{worst}} = \frac{9}{9} \times \frac{8}{8} \times \frac{6}{7} \times \frac{4}{6} \times \frac{2}{5} = \frac{384}{2520} \approx 0.152.$
- **Expected number of pairs** in the whole collection of ten socks is simply the number of actual pairs, i.e. **5**.

  (If the question asks for the expected number of *matched pairs* observed after each draw, the linearity-of-expectation argument yields the same value 5 when the process finishes.)

📏 Binary Representation Length

> *Definition*: The **bit-length** b of a positive integer n is the number of bits required to write n in binary without leading zeros.

- **Formula (2.1)**

  $b = \lfloor \log_2 n \rfloor + 1.$

  Proof sketch:

  1. $2^{b-1} \leq n < 2^b$ by definition of b .
  2. Taking $\log_2$ gives $b - 1 \leq \log_2 n < b$.
  3. Hence $b = \lfloor \log_2 n \rfloor + 1$.
- **Alternative formula**

  $b = \lceil \log_2(n+1) \rceil.$

Proof: From $2^{b-1} \leq n < 2^b$ add 1 to each part: $2^{b-1} + 1 \leq n + 1 \leq 2^b$. Since $2^{b-1} + 1 \leq 2^b$, taking $\log_2$ and the ceiling yields the expression above.

📈 Comparing Orders of Growth: $n!$ vs. 2^n

Using **Stirling's approximation** $n! \sim \sqrt{2\pi n}, n^n e^{-n}$:

$\lim_{n\to\infty} \frac{n!}{2^n} = \lim_{n\to\infty} \frac{\sqrt{2\pi n}, n^n e^{-n}}{2^n} = \lim_{n\to\infty} \frac{\sqrt{2\pi n}, n}{2e^{,n}} = \infty.$

> Therefore $n!$ grows **asymptotically faster** than 2^n .
> Symbolically: $2^n \in o(n!)$, and $n! \in \Theta(2^n)$ does **not** hold (the limit is infinite).

🏷️ Basic Asymptotic Efficiency Classes

Class	Name	Typical Example
1	**constant**	trivial loops that run a fixed number of times
$\log n$	**logarithmic**	binary search; repeatedly halving the problem size
n	**linear**	sequential search, scanning a list
$n \log n$	**linearithmic**	mergesort, quicksort (average case)
n^2	**quadratic**	bubble sort, matrix multiplication (naïve)
n^3	**cubic**	triple-nested loops, some linear-algebra algorithms
2^n	**exponential**	generating all subsets of an n-element set
$n!$	**factorial**	generating all permutations of an n-element set

Note: Constants hidden by O, Θ, Ω are ignored; nevertheless, for realistic input sizes the class with the lower growth rate usually dominates performance.

🔍 Sequential Search – Asymptotic Notation

Case	Notation	Reason
Worst	$O(n)$	In the worst case the key is absent, requiring n comparisons.
Best	$\Omega(1)$	The key may be found at the first position, needing only one comparison.
Average	$\Theta(n)$	Expected number of comparisons is $(n+1)/2$, which is linear.

📊 Summation Formulas

1. Closed-form sums

Sum	Result
$1+3+5+\cdots+999$	$250{,}000$
$2+4+8+\cdots+1024$	2046
$\sum_{i=3}^{n+1} 1$	$n-1$
$\sum_{i=3}^{n+1} i$	$\frac{(n+1)(n+2)}{2}-3$
$\sum_{i=0}^{n-1} i(i+1)$	$\frac{n(n-1)(n+1)}{3}$

$\sum_{j=1}^{n}(3j+1)$	$\frac{3n(n+1)}{2}+n$
$\sum_{i=1}^{n}\sum_{j=1}^{i} ij$	$\frac{n^2(n+1)^2}{4}$
$\sum_{i=1}^{n}\frac{1}{i(i+1)}$	$1-\frac{1}{n+1}$

2. Orders of growth for the indicated sums

Sum	Growth class $\Theta(\cdot)$
$\sum_{i=0}^{n-1}(i^2+1)^2$	$\Theta(n^5)$
$\sum_{i=2}^{n-1}\lg i^2$	$\Theta(n\log n)$
$\sum_{i=1}^{n}(i+1)^{2i-1}$	$\Theta!\left((n+1)^{2n-1}\right)$ (dominant last term)
$\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}(i+j)$	$\Theta(n^3)$

📐 Counting Operations for Sample Variance

Two algebraically equivalent formulas:

1. **Direct definition**

$$s^2=\frac{\sum_{i=1}^{n}(x_i-\bar{x})^2}{n-1},\quad \bar{x}=\frac{\sum_{i=1}^{n}x_i}{n}$$

2. **Computational shortcut**

$$s^2 = \frac{\sum_{i=1}^{n} x_i^2 - \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n}}{n-1}$$

Operation	Formula 1	Formula 2
Divisions	2 (one for $\bar{x}$, one for final division)	2 (division by n inside the square, final division)
Multiplications	n (computing each $(x_i - \bar{x})^2$) + 1 (square of $\bar{x}$) ≈ $n+1$	n (computing x_i^2) + n (product of $\sum x_i$ with itself) ≈ $2n$
Additions/Subtractions	n (computing $x_i - \bar{x}$) + n (squaring additions) + $n-1$ (sum) ≈ $3n-1$	$n-1$ (sum of x_i^2) + $n-1$ (sum of x_i) + 1 (final subtraction) ≈ $2n-1$

Conclusion: The shortcut reduces the number of subtractions but roughly doubles the multiplications; which formula is faster depends on the relative cost of these operations on the target hardware.

🧩 Analyses of Selected Algorithms

🔎 Mystery

```
ALGORITHM Mystery(n)
   S ← 0
   for i ← 1 to n do
       S ← S + i * i
   return S
```

- **Computes**: $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$
- **Basic operation**: multiplication $i * i$ (or addition to S).
- **Count**: executed n times → $\Theta(n)$.

- **Improvement**: use the closed-form formula above; the algorithm becomes $O(1)$ (constant time) after evaluating three multiplications and two additions.

🔢 Binary-Representation Recursion (BinRec)

- **Claim**: The recursive algorithm that repeatedly halves n performs exactly $\lfloor \log_2 n \rfloor$ additions.
- **Proof idea**: Each recursive call reduces the argument to $\lfloor n/2 \rfloor$. The recursion depth stops when $n = 1$, which occurs after $\lfloor \log_2 n \rfloor$ halvings. One addition is performed per level.
- **Non-recursive recurrence**

  Let $A(n)$ be the number of additions in the iterative version.

  $A(1) = 0, \qquad A(n) = A!\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1.$

  Solving yields $A(n) = \lfloor \log_2 n \rfloor$.

📈 Recursive Power of Two

```
ALGORITHM PowTwo(n)
   if n = 0 then return 1
   else return PowTwo(n-1) + PowTwo(n-1)
```

- **Recurrence for additions**: $T(0) = 0, T(n) = 2, T(n-1) + 1$.
- **Solution**

🔧 Transform-and-Conquer Techniques

📊 Frequency List (Mode) – Brute-Force

> **Definition** – A *frequency list* stores each distinct element of the input together with its occurrence count.

- Process each element A[i] of the original list.
- Scan the auxiliary list of distinct values seen so far:
  - If a match is found, increment its frequency.

    - Otherwise, insert the new value with frequency 1.
- **Worst-case input:** all elements are different.
- Number of comparisons:

$$C(n) = \sum_{i=1}^{n}(i-1) = 0 + 1 + \cdots + (n-1) = \frac{(n-1)n}{2} \in O(n^2).$$

- An additional (n-1) comparisons are needed to locate the maximum frequency; this does **not** change the quadratic class.

📈 Sorting-First Approach – (O(n\log n))

Sorting makes equal values contiguous, so the mode is the longest run of equal entries.

> **Key idea** – After sorting, a single linear scan finds the longest run.

```
ALGORITHM PresortMode(A[0..n-1])
   // Computes the mode of an array by sorting it first
   // Input:  array A[0..n-1] of orderable elements
   // Output: the array's mode
   sort the array A
   i ← 0                      // start of current run
   modefrequency ← 0
   while i ≤ n-1 do
       runlength ← 1
       runvalue ← A[i]
       while i + runlength ≤ n-1 and A[i + runlength] = runvalue
           runlength ← runlength + 1
       if runlength > modefrequency
           modefrequency ← runlength
           modevalue ← runvalue
       i ← i + runlength
   return modevalue
```

- The sorting step dominates the running time: (T_{\text{sort}}(n)=O(n\log n)).
- The subsequent scan is linear, so the overall worst-case efficiency is **(O(n\log n))**, a better asymptotic class than the brute-force (O(n^{2})).

🔍 Searching Revisited

- **Sequential search** on an unsorted array needs up to (n) comparisons (worst case).

- **Sort-then-binary-search**: sorting costs (O(n\log n)); binary search adds (O(\log n)) comparisons.
- Total worst-case time:

$$T(n) = O(n \log n) + O(\log n) = O(n \log n),$$

which is **worse** than plain sequential search for a single query. The same conclusion holds for average-case performance.

🧮 Gaussian Elimination & LU Decomposition

- Gaussian elimination consists of a **cubic elimination stage** (the "forward elimination") and a **quadratic back-substitution stage**.
- Counting the innermost loop executions gives (\Theta(n^{3})) multiplications (and the same number of subtractions), so the algorithm is **cubic**: (O(n^{3})).

> **LU decomposition** – Factor the coefficient matrix (A) as (A = LU), where

$$L = \begin{pmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}.$$

- Solving (Ax=b) becomes solving two triangular systems:
  1. **Forward substitution:** (Ly = b) (easy because (L) is lower-triangular).
  2. **Backward substitution:** (Ux = y) (easy because (U) is upper-triangular).

Modern implementations base Gaussian elimination on this LU factorisation.

🧩 Problem-Reduction Strategy

> **Definition** – *Problem reduction* means transforming an instance of one problem into an instance of another problem that is already known how to solve.

- **Synthetic division** reduces polynomial evaluation to Horner's rule.
- **Geometric orientation** reduces a spatial question about three points to the sign of a determinant.

- **Least Common Multiple (LCM):**

  $\operatorname{lcm}(m,n) = \frac{m \cdot n}{\gcd(m,n)}$,

  where (\gcd(m,n)) is obtained efficiently by Euclid's algorithm.

  This reduction avoids prime-factorisation and yields an (O(\log \min{m,n}))

  algorithm.

📈 Counting Paths with Matrix Powers

> **Theorem** – The number of distinct paths of length (k>0) from vertex (i) to vertex (j) in a (directed or undirected) graph equals the ((i,j)) entry of the matrix (A^{k}), where (A) is the adjacency matrix.

- Example adjacency matrix and its square:

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}, \qquad A^2 = \begin{pmatrix} 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 2 \\ 1 & 1 & 1 & 2 \end{pmatrix}.$$

- Entry (A^{2}*{aa}=3) indicates three length-2 paths that start and end at vertex (a); (A^{2}*{ac}=1) indicates a single length-2 path from (a) to (c).
- Computing (A^{k}) can be done with exponentiation-by-squaring, re-using the same fast-power techniques discussed for scalar powers.

🔐 Hashing Fundamentals

- **Dictionary ADT** – set supporting **search**, **insert**, **delete**.
- **Hash table** H[0..m-1] stores keys after applying a **hash function** (h) that maps each key to an integer in ([0,m-1]).

Key type	Sample hash function
Non-negative integer	(h(K)=K \bmod m)
Letter (alphabet)	(h(K)=\operatorname{ord}(K) \bmod m)
String (c_{0}c_{1}\dots c_{s-1}) (simple)	(\bigl(\sum_{i=0}^{s-1}\operatorname{ord}(c_{i})\bigr) \bmod

	m)
String (polynomial)	h ← 0; for i ← 0 to s-1 do h ← (h*C + ord(ci)) mod m

- **Desirable properties** of a hash function:
  - *Uniform distribution* → minimizes collisions.
  - *Efficiency* → cheap to compute (few arithmetic operations).
- **Collisions** occur when two distinct keys map to the same address; they are unavoidable when (m
- The table size (m) should be chosen to balance space usage and collision probability.

📚 Dynamic Programming

💰 Coin-Row Problem (recap)

- Recurrence:

$$F(i) = \max c_i + F(i-2),; F(i-1), \qquad F(0) = 0,; F(1) = c_1.$$

- Bottom-up computation fills an array F[0..n] in (O(n)) time and space.
- Back-tracing (or storing a "choice" array) recovers the optimal subset of coins.

💸 Change-Making Problem

- **Goal:** Minimum number of coins that sum to amount (n) using denominations (d_{1}
- Recurrence:

$$F(0) = 0, \qquad F(n) = 1 + \min_{j:,n \geq d_j} F(n - d_j) \quad (n > 0).$$

- Bottom-up algorithm:

```
ALGORITHM ChangeMaking(D[1..m], n)
   // D[1..m] are increasing denominations, D[1]=1
   // Returns the minimum number of coins that sum to n
   F[0] ← 0
   for amount ← 1 to n do
```

```
        best ← ∞
        for j ← 1 to m while D[j] ≤ amount do
            candidate ← F[amount - D[j]] + 1
            if candidate < best then best ← candidate
        F[amount] ← best
    return F[n]
```

- Time complexity: (O(n\cdot m)); space complexity: (O(n)).

📊 Warshall's Algorithm for Transitive Closure

> **Formula** – For (k=1\ldots n),

$ r^{(k)}_{ij}=r^{(k-1)}_{ij}\ \text{or}\ \bigl(r^{(k-1)}_{ik}\ \text{and}\ r^{(k-1)}_{kj}\bigr). $

- If there is already a path from (i) to (j) using intermediate vertices (\le k-1), the entry stays 1.
- Otherwise, it becomes 1 exactly when both (i\to k) and (k\to j) paths exist (using vertices (\le k-1)).

```
ALGORITHM Warshall(A[1..n,1..n])
   // Computes the transitive closure of a digraph
   // Input: adjacency matrix A
   // Output: matrix R = transitive closure
   R ← A
   for k ← 1 to n do
       for i ← 1 to n do
           for j ← 1 to n do
               R[i,j] ← R[i,j] or (R[i,k] and R[k,j])
   return R
```

- The algorithm runs in (O(n^{3})) time and (O(1)) extra space (in-place update).
- Example progression (excerpt) for a 4-vertex graph:

Step	Matrix (1 = path)
(R(0)) (adjacency)	(\begin{smallmatrix}0&0&0&1\1&0&0&0\1&
(R(1)) (through vertex a)	…

(R(4)) (final)	all reachable pairs marked 1 (see transcript).

🌲 Greedy Algorithms – Minimum Spanning Tree

- **Prim's algorithm** builds a MST by repeatedly adding the cheapest edge that connects a vertex inside the current tree to a vertex outside.
- **Connectivity check:** Prim's algorithm implicitly discovers that a graph is disconnected when the priority queue becomes empty before all vertices are reached.
- **Negative edge weights:** The algorithm remains correct; it always picks the minimum-weight crossing edge regardless of sign.
- **Updating a MST after adding a new vertex:** If a new vertex (v) is linked to the existing MST (T) by edges, the MST of the enlarged graph can be obtained by adding the cheapest edge from (v) to (T) (and then re-checking for possible cheaper replacements, but a single addition suffices when only one vertex is added).

🧩 Prim's Algorithm on an Unweighted Connected Graph (Q 13)

> **Definition** – A *minimum spanning tree* (MST) of a weighted connected graph is a spanning tree whose total edge-weight sum is minimum.

- If every edge has the same weight (e.g., weight = 1), Prim's algorithm still chooses the *nearest* vertex at each step, but "nearest" is no longer a discriminating criterion.
- The algorithm will arbitrarily pick any edge that connects the current tree to a new vertex; the result is simply **any** spanning tree, not a distinguished one.
- **Assessment:** Prim's algorithm works (it will produce a spanning tree) but offers **no advantage** over a trivial breadth-first or depth-first search for unweighted

graphs. Hence it is **not a good algorithm** for this particular problem.

🔒 Uniqueness of the MST with Distinct Edge Weights (Q 14)

> **Theorem** – In a connected graph whose edge weights are all *distinct*, the minimum spanning tree is **unique**.

Proof Sketch

1. Assume two different MSTs, T_1 and T_2 .
2. Let e be the smallest-weight edge that belongs to one tree but not the other; w.l.o.g. $e \in T_1 \setminus T_2$.
3. Adding e to T_2 creates a unique cycle C (since T_2 is a tree).
4. All edges of C have weight **greater** than $w(e)$ because e is the smallest edge not common to both trees.
5. Remove from C the edge f that is in T_2 but not in T_1 (such an f exists).
   The new tree $T_2' = T_2 \cup e \setminus f$ has total weight
   $w(T_2') = w(T_2) + w(e) - w(f) < w(T_2)$.
6. This contradicts the optimality of T_2 .
   Hence the assumption of two distinct MSTs is false, and the MST is unique.

🔧 Changing an Element's Value in a Min-Heap (Q 15)

> **Definition** – A *min-heap* is a binary tree where each node's key is ≤ the keys of its children, enabling $O(1)$ access to the minimum element.

Update Procedure

1. **Locate** the element whose key is to be changed (requires a pointer or index).
2. **Assign** the new key value.
3. If the new key is **smaller** than the old key, **bubble-up**:
   - Repeatedly compare the node with its parent; if the node's key is smaller, swap them.
4. If the new key is **larger**, **bubble-down** (heapify):
   - Repeatedly compare the node with its smaller child; if the child's key is smaller, swap them.

5. Stop when the heap property is restored.

Time Efficiency

- Each bubble step moves the element one level up or down the tree.
- The height of a heap with n elements is $\lfloor \log_2 n \rfloor$.
- Therefore the update runs in $O(\log n)$ time.

```
def heap_update(heap, i, new_key):
    old_key = heap[i]
    heap[i] = new_key
    if new_key < old_key:                # bubble-up
        while i > 0 and heap[i] < heap[parent(i)]:
            heap[i], heap[parent(i)] = heap[parent(i)], heap[i]
            i = parent(i)
    else:                                # bubble-down
        heapify_down(heap, i)
```

📚 Kruskal's Algorithm (Section 9.2) 🚀

> **Definition** – Kruskal's algorithm constructs an MST by repeatedly adding the next lightest edge that does **not** create a cycle.

Pseudocode (re-styled)

```
ALGORITHM Kruskal(G)
   // Input: weighted connected graph G =
   // Output: set ET of edges forming an MST

   sort E in nondecreasing order of weight
   ET ← ∅
   ecounter ← 0
   k ← 0
   while ecounter < |V| − 1 do
       k ← k + 1
       e ← the k-th edge in the sorted list
       if ET ∪ {e} is acyclic then
           ET ← ET ∪ {e}
```

```
            ecounter ← ecounter + 1
    return ET
```

Key Points

- The intermediate subgraph ET may be **disconnected**; only the final set of $|V| - 1$ edges is guaranteed to be a tree.
- **Cycle detection** is performed efficiently with a **union-find** (disjoint-set) data structure:
    - make-set(v) for each vertex,
    - find(v) to locate its component,
    - union(u, v) to merge components when an edge is added.
- **Correctness** follows the same cut-property argument used for Prim's algorithm.

Complexity

- Sorting edges: $O(|E| \log |E|)$.
- Union-find operations (with path compression & union by rank): almost linear, $O(|E|, \alpha(|V|))$, where α is the inverse Ackermann function.
- Overall: $O(|E| \log |E|)$, which dominates the runtime.

🚦 Dijkstra's Algorithm (Section 9.3) 🛤️

> **Definition** – Dijkstra's algorithm finds the shortest-path distances from a source vertex s to every other vertex in a weighted graph with **non-negative** edge weights.

Labels Used

- d_v: current best distance from s to vertex v (numeric label).
- p_v: predecessor of v on the current shortest path (parent label).

Pseudocode

```
ALGORITHM Dijkstra(G, s)
   Initialize priority queue Q
   for each vertex v in V do
       d[v] ← ∞ ; p[v] ← null
       Insert(Q, v, d[v])
   d[s] ← 0
```

```
    Decrease(Q, s, 0)
    VT ← ∅
    while Q not empty do
        u ← DeleteMin(Q)          // vertex with smallest d
        VT ← VT ∪ {u}
        for each neighbor v of u not in VT do
            if d[u] + w(u,v) < d[v] then
                d[v] ← d[u] + w(u,v)
                p[v] ← u
                Decrease(Q, v, d[v])
    return d, p
```

Time Efficiency

- Depends on the priority-queue implementation:
  - Binary heap: $O((|V| + |E|) \log |V|)$.
  - Fibonacci heap: $O(|V| \log |V| + |E|)$.
- The algorithm is **correct** because each extracted vertex u has its final shortest-path distance (no later improvement possible), a property proved by induction on the number of extracted vertices.

📚 Greedy Technique Summary (Recap)

Algorithm	Greedy Choice	Structure Built
Prim	Add the cheapest edge incident to the current tree	Growing *connected* tree
Kruskal	Add the cheapest edge that avoids a cycle	Growing *acyclic* subgraph (may be disconnected)
Dijkstra	Add the fringe vertex with smallest tentative distance	Growing *shortest-path* tree from source
Huffman	Merge two least-frequent symbols repeatedly	Building an optimal prefix-code tree

- Each step is **feasible**, **locally optimal**, and **irrevocable**.
- Correctness relies on *cut* or *exchange* arguments specific to the problem.

🔄 Iterative Improvement (Section 10)

> **Definition** – An *iterative-improvement* algorithm starts from a feasible solution and repeatedly applies a local change that improves the objective value, stopping when no improving move exists.

General Template

1. **Initialize** with a feasible solution (often the zero flow for max-flow problems).
2. **Search** for a *local improvement* (e.g., an augmenting path, a better matching edge).
3. **Apply** the improvement, obtaining a new feasible solution with a better objective.
4. **Repeat** until no improvement can be found; the final solution is then optimal.

Advantages & Challenges

- Requires a **neighbourhood definition** (what local changes are allowed).
- May get stuck in a **local optimum** that is not global; problem-specific proofs are needed to guarantee optimality.

🌊 Maximum-Flow Problem (Section 10.2) 📈

> **Definition** – A *flow network* is a directed graph $G = \langle V, E \rangle$ with a distinguished source s (no incoming edges) and sink t (no outgoing edges). Each edge (i, j) has a capacity $u_{ij} > 0$.

Flow Conservation & Capacity Constraints

- For every **intermediate** vertex i; $(2 \le i \le n-1)$:
  \sum_{j:(j,i)\in E} x_{ji};=;\sum_{j:(i,j)\in E} x_{ij} \tag{10.8}
- For each edge:
  0 \le x_{ij} \le u_{ij} \tag{10.10}
- **Flow value** v (to be maximized):
  v = \sum_{j:(s,j)\in E} x_{sj} = \sum_{j:(j,t)\in E} x_{jt} \tag{10.9}

Augmenting-Path (Ford–Fulkerson) Method

1. Start with the **zero flow** ($x_{ij} = 0$ for all edges).
2. While there exists a **flow-augmenting path** P from s to t in the *residual network* (edges with remaining capacity $u_{ij} - x_{ij} > 0$):
   - Let $\Delta = \min u_{ij} - x_{ij} \mid (i, j) \in P$.

    - Increase x_{ij} by Δ on forward edges of P and decrease by Δ on backward edges.
3. When no augmenting path remains, the current flow is **maximum**.

Edmonds–Karp Implementation

- Uses **Breadth-First Search** to find the *shortest* (fewest edges) augmenting path.
- **Complexity:** At most $O(nm/2)$ augmentations; each BFS costs $O(m)$ (adjacency-list representation).
  Total time: $O(nm^2)$, which simplifies to $O(nm)$ because $m \geq n - 1$.

Max-Flow / Min-Cut Theorem (informal)

- The value of a maximum flow equals the capacity of a minimum s–t cut.
- When the augmenting-path algorithm stops, the set of vertices reachable from s in the residual graph defines a minimum cut; all edges crossing from reachable to unreachable vertices are saturated (flow equals capacity).

🔗 Maximum Matching in Bipartite Graphs (Section 10.3)

> **Definition** – A *bipartite graph* $G = \langle V, U, E \rangle$ has vertex sets V and U with edges only between the sets. A *matching* $M \subseteq E$ pairs vertices such that no two edges share an endpoint.

Hall's Marriage Theorem

> A bipartite graph $G = \langle V, U, E \rangle$ has a matching that covers **all** vertices of V **iff** for every subset $S \subseteq V$, $|R(S)| \geq |S|$, where $R(S)$ is the set of neighbours of S in U .

- The theorem can be used as a **decision test** for a *perfect matching* on the V side.
- Directly checking the condition requires examining all $2^{|V|}$ subsets, which is exponential; practical algorithms avoid this enumeration.

Algorithmic Approach via Max-Flow

1. **Construct** a flow network:
    - Add a source s with edges (s, v) of capacity 1 for each $v \in V$.

    - Add a sink t with edges (u, t) of capacity 1 for each $u \in U$.
    - For each original edge $(v, u) \in E$, add a directed edge (v, u) of capacity 1.
2. **Compute** a maximum flow from s to t .
3. The flow value equals the size of a maximum matching.
4. Extract the matching: an edge (v, u) carries flow 1 ⇔ it belongs to the matching.
- Because capacities are integral, the Ford–Fulkerson (or Edmonds–Karp) algorithm yields an **integral** flow, directly giving a matching.

Complexity

- The constructed network has $|V| + |U| + 2$ vertices and $|E| + |V| + |U|$ edges.
- Using Edmonds–Karp: $O((|V| + |U|)(|E| + |V| + |U|))$, which is polynomial and sufficient for bipartite matching.

Example Checks (from the transcript)

| Set V | Condition $|R(S)| \geq |S|$? | |---------|----------------------------| | $1, 2, 3, 4$ | (checked against the graph in the transcript) | | $5, 6, 7$ | (checked against the graph in the transcript) |

(The actual adjacency lists are omitted; the table illustrates the required verification.)

📊 Summary of Greedy vs. Iterative-Improvement Strategies

Strategy	Typical Problem Type	Core Idea	Guarantees
Greedy	MST, shortest paths, Huffman coding	Build solution incrementally, always picking the locally best feasible element	Proven optimal when a *cut* or *exchange* property holds

Iterative Improvement	Maximum flow, bipartite matching, many NP-hard approximations	Start with a feasible solution; repeatedly apply a local improvement until none exist	Optimality proved via augmenting-path arguments (e.g., max-flow/min-cut) or reduction to a known optimal problem

These notes capture the essential algorithms, definitions, and theoretical results presented in the provided material, ready for integration into a larger study guide.

📊 Decision Trees for Sorting & Searching in a Sorted Array

> **Decision tree** – A binary tree that models the sequence of comparisons performed by a comparison-based algorithm. Each internal node represents a comparison, each leaf corresponds to a possible outcome (e.g., a permutation of the input).

- **Sorting**
  - A comparison-based sort can be seen as a decision tree whose leaves represent the $n!$ possible permutations of the n input keys.
  - The height h of any such tree must satisfy $2^h \geq n!$, hence $h \geq \lceil \log_2(n!) \rceil = \Omega(n \log n)$.
  - Example (three-element list a, b, c): the leaf labelled "$a < c < b$" corresponds to the permutation that places a first, c second, b third.
- **Searching in a sorted array**
  - Binary search on a sorted array of size n also induces a decision tree, but now the leaves correspond to the $n + 1$ possible outcomes (found at position i or not found).
  - The tree height is at most $\lceil \log_2(n + 1) \rceil$, giving the classic $O(\log n)$ worst-case time.

Building on the earlier discussion of algorithmic efficiency, decision-tree analysis provides a clean way to prove lower bounds for comparison-based problems.

🧩 Complexity Classes: P, NP, NP-Complete

> **Decision problem** – A problem with a yes/no answer; the standard model for studying computational complexity.

- **Class P** – All decision problems solvable by a deterministic algorithm in polynomial time.
- **Class NP** – Decision problems whose *proposed* solutions can be verified in polynomial time by a deterministic algorithm.
- **NP-complete** – A problem Q is NP-complete if
  1. $Q \in \mathrm{NP}$, and
  2. every problem $R \in \mathrm{NP}$ can be reduced to Q by a polynomial-time transformation.
     S. Cook proved the first NP-complete problem (CNF-satisfiability).
- The open question **P vs NP** asks whether every efficiently verifiable problem is also efficiently solvable; it is unknown whether $\mathrm{P} = \mathrm{NP}$ or $\mathrm{P} \subsetneq \mathrm{NP}$.

📦 Approximation Algorithms for NP-Hard Problems

Greedy algorithm for the (0/1) knapsack

1. Sort items by decreasing value-to-weight ratio v_i/w_i .
2. Scan the list, **select** an item if it fits; otherwise **skip** it.

- **Example showing unbounded error**

item	weight w_i	value v_i	v_i/w_i
1	1	2	2
2	W	W	1

Capacity $W > 2$. Greedy picks item 1 (value 2) and skips item 2, yielding total value 2. The optimal solution is item 2 alone (value W).
Approximation ratio $r = \dfrac{W}{2}$, which grows without bound as W increases.

Enhanced greedy algorithm with a guaranteed performance ratio

- **Procedure**:
    1. Run the greedy algorithm above, obtaining set S_{greedy} .
    2. Identify the single feasible item of maximum value, call it S_{single} .
    3. Return the better of S_{greedy} and S_{single} .
- **Performance guarantee**: The returned set S_{alg} satisfies

$\text{OPT} \leq 2; !|S_{\text{alg}}|$

and the factor 2 is tight (no smaller constant works for all instances).

Continuous knapsack (fractional) – optimal greedy

> **Continuous knapsack** – Items may be taken in arbitrary fractions; the goal is to maximize total value subject to the capacity constraint.

Greedy algorithm

1. Compute v_i/w_i for each item.
2. Sort items in non-increasing order of these ratios.
3. Scan the sorted list:
    - If the whole item fits, take it.
    - Otherwise take the largest possible fraction of the current item to fill the knapsack and **stop**.
- **Optimality argument**: Because items are processed by decreasing efficiency, any solution that does not fill the knapsack with the current highest-ratio item can be improved by swapping a portion of a lower-ratio item for a portion of the higher-ratio one. Hence the greedy construction yields the maximum possible total value.

📈 Root-Finding Methods

🔧 Method of False Position (Regula Falsi)

> **Regula falsi** – An interpolation-based bracketing method for locating a root of a continuous function f that changes sign on $[a_n, b_n]$.

- **Iteration formula**

$x_n = \frac{a_n, f(b_n) - b_n, f(a_n)}{f(b_n) - f(a_n)}$

(the x-intercept of the line through $(a_n, f(a_n))$ and $(b_n, f(b_n))$).

- **Comparison to bisection**: Like bisection, each iteration maintains a sign-changing interval, but the next approximation is not the midpoint; it is often nearer to the root, giving faster convergence on many problems.
- **Illustrative trace** – The first eight iterations for a particular cubic equation (see Figure 12.21 in the source) show the method converging, though not as rapidly as Newton's method for that example.

🧮 Newton's Method (Newton-Raphson)

> **Newton's method** – An iterative scheme that uses the tangent line at the current approximation to predict the next one.

- **Update rule**

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \qquad (n = 0, 1, \dots)$$

- **Typical convergence**: If the initial guess x_0 is "close enough" to a simple root $x^andf'(x)doesnotvanishnearx^$, the sequence converges **quadratically** (error roughly squares each step).
- **Example – square root**: Solving $x^2 - a = 0$ gives

$$x_{n+1} = \frac{1}{2}!\left(x_n + \frac{a}{x_n}\right),$$

the classic iteration for $\sqrt{a}$.

- **Example – cubic equation** $x^3 - x - 1 = 0$

$$x_{n+1} = x_n - \frac{x_n^3 - x_n - 1}{3x_n^2 - 1}.$$

Starting from $x_0 = 2$, the first five iterates (see Figure 12.23) rapidly approach the root, dramatically outpacing both bisection and false-position.

- **Practical considerations**
  - Each iteration requires evaluating both f and f' .

- The method does **not** guarantee a bracketing interval; divergence is possible if x_0 is far from any root or if $f'(x)$ becomes zero.
- A useful stopping criterion derives from the Mean Value Theorem: if $|f'(x)| \geq m_1 > 0$ on the interval containing the root, then

$|x_n - x^*| \leq \frac{|f(x_n)|}{m_1}$.

```
Hence one may stop when $\dfrac{|f(x_n)|}{m_1}<\varepsilon$ .
```

- Alternative criteria: $|x_n - x_{n-1}| < \varepsilon$ or $|f(x_n)| < \varepsilon$.

📚 References

- Gonnet & Baeza-Yates, *Handbook of Algorithms and Data Structures*, 1991.
- Goodrich & Tamassia, *Algorithm Design*, 2002.
- Graham, Knuth & Patashnik, *Concrete Mathematics*, 1994.
- Green & Knuth, *Mathematics for Analysis of Algorithms*, 2007.
- Additional citations list classic works on complexity, approximation, and numerical methods as provided in the source material.